TalentScout — AI Hiring Assistant

This document provides a detailed explanation of the TalentScout Hiring Assistant project, developed as part of the AI/ML Intern Assignment. The assistant is designed to act as an initial screening chatbot for a recruitment agency, collecting candidate details and generating tailored technical questions based on the declared tech stack.

1. Project Overview

TalentScout is a Streamlit-based chatbot application that interacts with candidates in a recruitment context. It gathers essential information such as personal details, years of experience, desired position, and technical skills. Based on the provided tech stack, it generates relevant technical interview questions using either an OpenAI API (if available) or a curated fallback question bank.

2. Key Features

- Clean and intuitive UI built with Streamlit
- Greeting and introduction for candidates
- Candidate form for information gathering
- Tech-stack declaration and parsing
- Automatic generation of 3–5 technical questions per technology
- Fallback mechanism if LLM is unavailable
- Local persistence of candidate details in CSV file
- Chatbox for free-text interaction and graceful conversation ending
- Demo dataset with pre-populated candidates
- Download option for saved candidates

3. Architecture and File Structure

The project is modular and follows a simple architecture for clarity and maintainability.

```
talentscout-chatbot/
│── app.py            # Streamlit UI, forms, chat, saved candidates
│── chatbot.py        # Logic for generating technical questions
│── prompts.py        # Predefined prompt templates for LLM
│── utils.py          # Helper functions for validation, persistence
│── requirements.txt  # Project dependencies
│── .env.example      # Environment variable template
│── data/candidates.csv  # Saved candidate records (demo data included)
```

```
│── README.md          # Documentation
```

4. Technical Details

Programming Language: Python
Frontend: Streamlit
Backend: OpenAI API (optional), fallback Q-bank otherwise
Data Storage: Local CSV file in data/candidates.csv
Environment: Virtual environment with dependencies listed in requirements.txt

The application uses environment variables from .env for secure handling of sensitive keys. If no OpenAI API key is set, the chatbot defaults to a curated fallback bank of questions.

5. Prompt Engineering

Prompts are defined in prompts.py. Three key prompts are used:
1. SYSTEM_PROMPT — defines chatbot behavior, scope, tone, and exit rules
2. QUESTION_PROMPT — instructs the model to output structured JSON with interview questions
3. FALLBACK_PROMPT — requests minimal JSON when LLM cannot handle unknown technology

6. Data Handling and Privacy

Candidate details are collected through Streamlit form fields and stored locally in a CSV file. Demo records are included to simulate functionality. A privacy disclaimer is shown in the app explaining that this is for demonstration only. For production, encryption, consent, and GDPR/DPDP compliance would be required.

7. User Interaction Flow

1. Candidate opens the chatbot and sees greeting + privacy note
2. Candidate fills in details (name, email, phone, YOE, etc.)
3. Candidate specifies tech stack (comma-separated)
4. Clicks 'Generate Questions' → tailored interview questions appear
5. Clicks 'Save Candidate Record' → saved in data/candidates.csv
6. Candidate can view saved records in-app under 'View Saved Candidates'
7. Candidate can chat briefly with the bot, and exit with keywords like 'bye' or 'exit'

8. Deliverables

- Complete codebase with app.py, chatbot.py, prompts.py, utils.py
- requirements.txt and .env.example for reproducibility
- README.md with setup, usage, and technical details
- Pre-populated data/candidates.csv with demo records
- LOOM_SCRIPT.md for recording video demo

9. Challenges and Solutions

- Issue: Handling unknown technologies → Solution: fallback generic prompts + curated Q-bank
- Issue: Streamlit rerun deprecation → Solution: replaced experimental_rerun with st.rerun
- Issue: Missing dependency resolution in VS Code → Solution: ensured venv activation and installed packages

10. Evaluation Criteria Mapping

- Technical Proficiency — All chatbot functionalities implemented
- Problem-Solving — Effective prompt design, fallback handling
- UI/UX — Clear Streamlit interface
- Documentation — README.md, this project doc
- Optional Enhancements — Demo dataset, Saved Candidates view, CSV download

11. Conclusion

TalentScout is a fully functional prototype of an AI-powered Hiring Assistant chatbot. It demonstrates integration of LLM prompt engineering, user interaction design with Streamlit, and basic data persistence. The project meets all assignment requirements and includes documentation and a demo script for evaluation.